MINI PROJECT REPORT

On

"EDUVAULT: DIGITAL LOCKER AND ACADEMIC CREDENTIAL VERIFICATION PLATFORM"

By

Akshay Darji — PRN: **1272250311**

Priyesh Nandanwar — PRN: **1272250279**

For

MCA40130 Mini Project
SY MCA (Semester III)
AY 2026-2027

Under The Guidance Of

Mrs. Sukanya Akolkar

Submitted To

DR. VISHWANATH KARAD
MIT WORLD PEACE UNIVERSITY, PUNE

In partial fulfillment of the requirement for the award of Degree of
Master of Computer Applications (MCA)

Submitted Through
School of Computer Science and Engineering
Department of Computer Science and Applications
MIT-WPU, Pune - 411038

Dr. Vishwanath Karad MIT WORLD PEACE UNIVERSITY, PUNE

School of Computer Science and Engineering

Department of Computer Science and Applications

CERTIFICATE

This is to certify that, **Akshay Darji** (PRN: **1272250311**) and **Priyesh Nandanwar** (PRN: **1272250279**), students of **SY MCA, Semester III** have completed Mini Project entitled **"EduVault: Digital Locker and Academic Credential Verification Platform"** under guidance of **Mrs. Sukanya Akolkar** in partial fulfilment of requirement for the award of **Master of Computer Applications** prescribed under **Dr. Vishwanath Karad MIT World Peace University, Pune**.

This project is the record of authentic work carried out by them during the academic year **2026-2027**.

Mrs. Sukanya Akolkar
Internal Guide

Dr. P. S. Metkewar
Program Head - MCA

Dr. Anuradha S. Kanade
Program Director

Date: 28 / 10 / 2026 **Place:** Pune

ACKNOWLEDGEMENT

We wish to express our profound gratitude to all the individuals who supported us throughout the completion of our Mini Project III. This report is a testament to their guidance, patience, and resources.

We are deeply indebted to our esteemed project guide, **Mrs. Sukanya Akolkar**, for her invaluable mentorship, direction, and continuous encouragement. Her advice was crucial in defining the scope, architectural framework, and methodology of this project.

We extend our sincere thanks to the Head of the Program, **Dr. P. S. Metkewar**, and the Program Director, **Dr. Anuradha S. Kanade**, for providing the necessary computing facilities, academic framework, and environment conducive to advanced project work and research.

Our appreciation also goes to all the **Faculty Members of the Department of Computer Science and Applications** for their comprehensive instruction and expertise, which equipped us with the knowledge foundational to this project.

We thank our fellow Project Group Members, for their unwavering dedication, collaborative spirit, and commitment to shared goals. This project is a product of our collective effort.

Finally, we are grateful to our **Classmates** for their constructive feedback and discussions, which enriched our understanding and helped us refine our approach.

Field	Team Member 1	Team Member 2
Name:	Akshay Darji	Priyesh Nandanwar
PRN:	1272250311	1272250279
Signature:		

Date: 28 / 10 / 2026 **Place:** Pune

DECLARATION

We, **Akshay Darji** and **Priyesh Nandanwar**, on behalf of our team hereby declare that this Mini Project report entitled **"EduVault: Digital Locker and Academic Credential Verification Platform"** is an authentic record of our own work carried out under the supervision of **Mrs. Sukanya Akolkar**.

We further declare that the content of this report, including all design, coding, testing, and analysis, is original and has not been submitted previously for any other degree or diploma at this or any other institution.

We take sole and full responsibility for any errors, omissions, or misrepresentations found within this report.

Field	Team Member 1	Team Member 2
Name:	Akshay Darji	Priyesh Nandanwar
PRN:	1272250311	1272250279
Signature:		

TABLE OF CONTENTS

1. INTRODUCTION 4
1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM 4
1.1.1 Existing System 4
1.1.2 Proposed System 5
1.1.3 Comparative Analysis 6
1.1.4 Need for the System 6
1.1.5 Objectives of the System 7
1.2 SCOPE OF WORK 8
1.2.1 Functional Scope 8
1.2.2 Technical Scope 8
1.2.3 Out of Scope and Boundaries 9
1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE 9
1.3.1 Hardware Requirements 9
1.3.2 Software Requirements 10
1.4 Description of Technology used 11
1.4.1 Python 3.11 Runtime Environment 11
1.4.2 Django 6.0 and Django REST Framework (DRF) 11
1.4.3 SimpleJWT Token Authentication Architecture 11
1.4.4 React 18 and Vite Tooling Engine 11
1.4.5 Tailwind CSS and Lucide React Visual Components 11
1.4.6 PostgreSQL 16 and Neon Serverless Persistence 12
1.4.7 QRCode.react and Cryptographic Token Verification 12
1.4.8 Gunicorn WSGI and WhiteNoise Production Static Server 12
2. PROPOSED SYSTEM 13
2.1 FEASIBILITY STUDY 14
2.1.1 Technical Feasibility 14
2.1.2 Operational Feasibility 14
2.1.3 Economic Feasibility 14
2.1.4 Feasibility Conclusion 14
2.2 USER REQUIREMENTS 15
2.2.1 User Classes and Actor Profiles 15
3. SYSTEM DESIGN 16

3.1 Object Diagram 16
3.2 Class Diagram 17
3.3 Use Case Diagram 18
3.4 Sequence Diagram 18
3.5 Activity Diagram 19
3.6 Module Hierarchy Diagram 20
3.7 USER INTERFACE DESIGN 20
3.7.1 UI Architecture Philosophy 20
3.7.2 Navigation and State Diagram 21
3.7.3 Screen Inventory 22
3.8 TABLE SPECIFICATIONS 22
3.8.1 Database Architecture Rationale 22
3.8.2 Entity-Relationship Diagram 23
3.8.3 Data Dictionaries 24

TABLE OF CONTENTS (CONTD.)

4. INPUT AND OUTPUT SCREENS 26
4.1 INPUT SCREENS 26
4.1.1 Authentication and Login Screen (/login) 26
4.1.2 User Registration Screen (/register) 27
4.1.3 Digital Locker Document Upload Screen (/certificates) 28
4.1.4 Placement Candidate Filter Screen (/placement) 29
4.2 OUTPUT SCREENS 30
4.2.1 Student Digital Locker Dashboard (/dashboard) 30
4.2.2 Interview Key & QR Code Modal View 31
4.2.3 Academic Committee Review Queue (/committee/documents) 32
4.2.4 Dynamic Alumni Network Directory (/alumni) 33
4.2.5 Public Candidate Verification Dossier (/student/:id) 35

5. TEST PROCEDURES AND IMPLEMENTATION 37
5.1 IMPLEMENTATION STRATEGY 37
5.2 TEST STRATEGY AND TESTING ENVIRONMENT 37
5.2.1 Testing Levels 37
5.2.2 Test Environment 38
5.3 TEST CASES AND TEST RESULTS 38
5.4 DEFECTS FOUND AND RESOLVED 40
5.4.1 Defect 1: Production Media File 404 Resolution (DEBUG=False) 40
5.4.2 Defect 2: Monorepo Root Directory & SPA Rewrites on Vercel 41

6. SAMPLE CODE 42
6.1 Custom User & Role-Based Permissions (accounts/models.py) 42
6.2 Document Validation & Model Architecture (documents/models.py) 43
6.3 Committee Verification View (documents/views.py) 45
6.4 Frontend Dynamic Media URL Resolver (src/api/apiClient.js) 47
Explanation & Architecture Mechanics 48

7. LIMITATIONS 50
7.1 Cloud Free-Tier Spin-Down Boundary 50
7.2 Ephemeral File Storage on Free Containers 50
7.3 Manual Committee Review Scalability 50
7.4 Operational and Evaluation Limits 51

8. FUTURE SCOPE 52

8.1 Blockchain Smart Contracts (Ethereum & Polygon Integration) 52

8.2 AI-Powered OCR for Automatic Credential Verification 52

8.3 Government DigiLocker and National Academic Depository (NAD) APIs 52

8.4 Mobile Native Applications via React Native 52

9. CONCLUSION 54

9.1 Technical Summary 54

9.2 Academic and Professional Reflection 54

10. REFERENCES 55

EduVault | Mini Project Report

1. INTRODUCTION

Higher education institutions generate enormous volumes of academic records, including degree certificates, semester marksheets, co-curricular awards, industrial internship validations, and technical project portfolios. However, the management, storage, verification, and dissemination of these records have historically remained fragmented across manual paper archives, disparate department files, and unverified digital scans. This fragmentation causes severe delays in institutional background checks, fosters academic fraud, burdens placement departments with manual spreadsheet compilation, and isolates graduated alumni from active university mentorship networks.

This chapter describes existing credential management practices and their limitations, introduces the proposed **EduVault** ecosystem, defines the project scope, outlines hardware and software operational environments, and justifies the technology selections powering the platform.

1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM

1.1.1 Existing System

Academic credential management and campus placement coordination currently depend on four loosely coupled, manual procedures:

(a) Physical Paper Transcripts & Unauthenticated PDF Scans.
Students submit physical photocopies or generic PDF scans of their credentials during semester evaluations and campus placement registrations. These files are not cryptographically tied to an institutional source of truth. Consequently, they are vulnerable to alteration, forged seals, and manipulated marks using basic graphic editing tools.

(b) Manual Administrative Verification Workflows.
Institutions establish academic verification committees to authenticate student credentials. However, verifiers must manually retrieve physical ledger books or query disconnected database spreadsheets. This creates heavy administrative backlogs, with third-party background verification agencies frequently waiting two to four weeks for manual transcript confirmations.

(c) Fragmented Campus Placement Compilation.
Training & Placement Officers (TPO) spend countless hours compiling candidate data (CGPA, active backlogs, departmental affiliations, and project links) through Google Forms or spreadsheets. This self-reported data is prone to human error, lacks instant verification, and cannot be dynamically presented to corporate recruiters.

(d) Static Alumni Communication Silos.

Upon graduation, alumni contact records deteriorate into static contact sheets. Universities lack an interactive, database-backed registry where alumni can advertise current company designations, share locations, and declare availability for student mentorship.

EduVault | Mini Project Report

1.1.2 Proposed System

EduVault is an end-to-end, enterprise-grade Digital Locker, Academic Credential Verification, and Career Networking Platform. It unites students, verification committees, placement officers, alumni, and external recruiters within a secure, role-governed web application. The platform operates across six core functional tiers:

1. **Authentication & Role Governance Tier:** Issues signed JSON Web Tokens (JWT) using Django REST Framework SimpleJWT, routing users automatically into one of four tailored portals: `STUDENT`, `ALUMNI`, `COMMITTEE`, or `PLACEMENT`.
2. **Student Digital Vault:** Enables students to upload, categorize, and track semester marksheets, degree certificates, and achievement files (≤ 5 MB). Uploaded records initiate in a `PENDING` state.
3. **Academic Committee Verification Pipeline:** Provides faculty reviewers with a real-time review queue, in-app full-screen document previewers, instant verification approvals, or rejection feedback mechanisms.
4. **Cryptographic Interview Key & QR Engine:** Generates unique UUID-based cryptographic endpoints (e.g. `/student/:id` and `/verify/:id`) that render as high-resolution QR codes, enabling corporate recruiters to scan and inspect verified student credentials on mobile devices instantly.
5. **Placement & Corporate Relations Dashboard:** Empowers placement teams with aggregate talent metrics (Total Students, Verified Ratio, Average CGPA), instant text search, and multi-tier CGPA threshold pills (≥ 9.0, ≥ 8.0, ≥ 7.5, ≥ 7.0).
6. **Dynamic Alumni Network:** An interactive, database-backed directory allowing graduated alumni to publish current roles, employers, geographic locations, and mentorship availability tags.

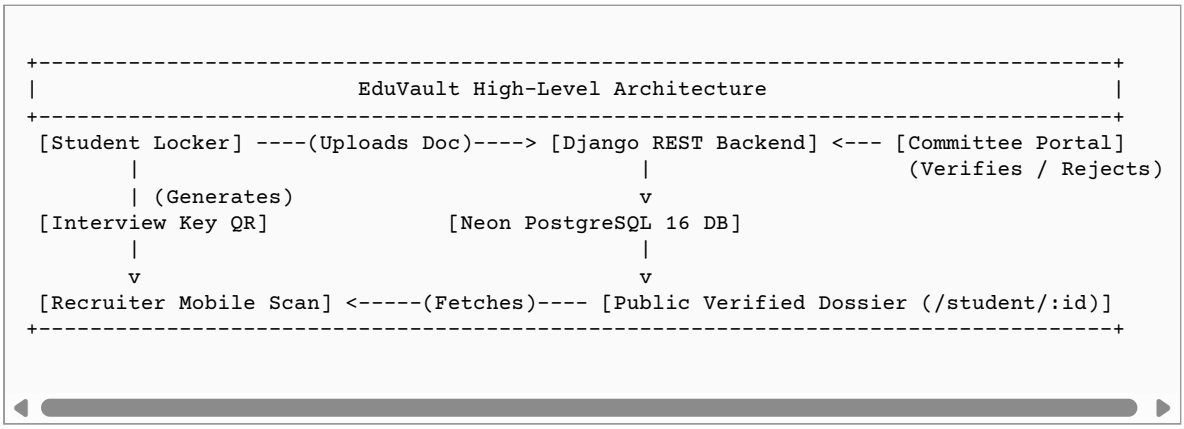


Figure 1.1: High-level architectural dataflow of the EduVault platform.

1.1.3 Comparative Analysis

Table 1.1 compares traditional institutional document management with EduVault across eight operational parameters.

Table 1.1: Comparison of existing approaches with EduVault.

Sr. No	Operational Parameter	Manual Paper / File Transcripts	Generic Cloud Drive (Google Drive)	EduVault Platform
1	**Verification Mechanism**	Manual stamp, physical ledger lookup	Unverified link sharing; no institutional approval	Formal academic committee pipeline with immutable verification timestamps
2	**Fraud Resistance**	Low; easily forged with photocopies & tools	Zero; raw uploaded PDFs cannot be trusted	High; UUID tokens and institutional QR verification ensure authenticity
3	**Recruiter Inspection**	Requires physical documents at interview	Requires manual downloading & unorganized viewing	1-click QR Code scan directly to mobile-ready verified dossier
4	**Role Governance**	None; unorganized file cabinets	Basic link permissions (Viewer/Editor)	Strict 4-Role RBAC: Student, Alumni, Committee, Placement Team
5	**Placement Analytics**	Manual spreadsheet tallying	Disconnected sheets prone to student typos	Live talent pool metrics, dynamic CGPA threshold filters, and verified pills
6	**Alumni Integration**	Static address books	None; personal LinkedIn networks only	Dynamic database registry with mentorship filters and company badges
7	**Document Lifecycle**	Uncontrolled paper lifecycle	Unmonitored file edits	State machine: PENDING → VERIFIED or REJECTED with feedback
8	**Hosting & Licensing Cost**	High physical paper & courier archiving costs	Paid workspace licenses	100% Free Production Deployment (Vercel + Render + Neon PostgreSQL)

1.1.4 Need for the System

Eliminating Academic Credential Fraud: With increasing competition in campus placements, recruiters face fabricated resumes and altered CGPA reports. EduVault eliminates fraud by anchoring credentials to institutional committee verification.

Instant Verification Latency Reduction: Employers spend up to two weeks on third-party verification. Scanning an EduVault Interview Key reduces verification latency to under 3 seconds.

Institutional Digital Transformation: Aligns the university with paperless, sustainable e-governance standards while providing students with a permanent digital portfolio.

EduVault | Mini Project Report

1.1.5 Objectives of the System

The engineering objectives are formulated with measurable, verifiable criteria as evaluated during system testing.

Table 1.2: Engineering objectives and verification criteria.

ID	Objective	Verification Criterion
O1	Implement a decoupled full-stack architecture with Django REST Framework backend and React 18 frontend.	Frontend and backend communicate purely over JSON REST APIs with no server-side HTML rendering.
O2	Establish Role-Based Access Control (RBAC) across four user classes: Student, Alumni, Committee, and Placement Team.	Role permissions are strictly enforced on backend endpoints (HTTP 403 Forbidden on unauthorized requests) and frontend route guards.
O3	Develop a Digital Locker allowing document uploads with strict size and extension validation.	Uploads ≤ 5 MB in PDF, JPG, and PNG succeed; oversized or executable files are rejected with HTTP 400.
O4	Build an Academic Committee review pipeline with approve and reject actions.	Committee reviews update document state to `VERIFIED` (with timestamp) or `REJECTED` with mandatory student feedback.
O5	Generate dynamic cryptographic QR Codes for physical and digital interview verification.	Mobile camera scan of the generated QR code opens the public dossier (`/student/:id`) with verified records on mobile browsers.
O6	Create a Talent Discovery Portal for Placement Officers with multi-criteria filtering.	Placement dashboard renders dynamic talent pool metrics and correctly filters candidates by CGPA thresholds (e.g. ≥ 8.0).
O7	Deploy the full-stack system to free production cloud tiers without hosting expenditures.	Frontend is live on Vercel Edge CDN, backend on Render WSGI service, and database on Neon Serverless PostgreSQL.
O8	Provide authenticated media file serving in production environments.	Uploaded certificates open cleanly from the live Render backend media endpoint with zero 404 routing errors.

1.2 SCOPE OF WORK

1.2.1 Functional Scope

The functional scope encompasses all user-facing interactions across the platform:

- **User Registration & Profiling:** Account creation with department, college, PRN, phone, and role assignment.
- **JWT Session Security:** Token issuance, automated refresh token rotation via Axios interceptors, and logout handling.
- **Student Vault Management:** Uploading, previewing, and deleting unverified academic documents.
- **Institutional Committee Queue:** Queue sorting, filtering by status/category, full-screen document inspection, single-click verification, and rejection with feedback.
- **Recruitment & Placement Engine:** Live talent metrics, candidate search, CGPA threshold filtering, and verified dossier export.
- **Dynamic Alumni Network:** Real-time alumni registry with employer, role, location, and mentorship availability filters.
- **Public QR Verification:** Tokenized, unauthenticated read-only candidate dossier view for external corporate recruiters.

1.2.2 Technical Scope

- **Backend REST Service:** Python 3.11 with Django 6.0 and Django REST Framework, served via Gunicorn WSGI.
- **Persistence:** PostgreSQL 16 (Neon Serverless Cloud) managed through Django ORM with ACID-compliant transactions and foreign key cascades.
- **Frontend Application:** React 18 with Vite build engine, React Router DOM v6, Tailwind CSS, Lucide React icons, and QRCode.react.
- **Cloud Infrastructure:** Vercel for SPA hosting with edge routing; Render for Python WSGI container execution.

1.2.3 Out of Scope and Boundaries

1. **Dynamic Analysis & OCR:** The system stores and displays files; automatic character recognition (OCR) of scanned certificates is excluded from the current release.
2. **Automated Grading:** The platform tracks CGPA and marksheets; it does not compute semester examination grades from scratch.
3. **Payment Gateway Integration:** Administrative transcript fees are handled externally; online financial transactions are out of scope.

4. **Native Mobile Apps:** The system is built as a fully responsive Web App; native compilation for Android APK or iOS IPA is not implemented.

EduVault | Mini Project Report

1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE

1.3.1 Hardware Requirements

Table 1.3 details the hardware specifications across development, production, and client access environments.

Table 1.3: Hardware requirements by deployment role.

Role	Component	Minimum Specification	Recommended Specification
Development Workstation	Processor	Dual-Core 64-bit CPU, 2.0 GHz	Intel Core i5 / AMD Ryzen 5, 2.8 GHz or higher
	RAM	8 GB DDR4	16 GB DDR4/DDR5
	Storage	256 GB SSD (500 MB free for project)	512 GB NVMe SSD
	Network	10 Mbps Broadband	50 Mbps Wi-Fi / Ethernet
Cloud Production Server	Compute	0.5 vCPU (Render Free Tier)	1.0 vCPU Dedicated Container
	RAM	512 MB RAM	1 GB RAM or higher
	Database Storage	0.5 GB Serverless PostgreSQL (Neon)	5 GB Persistent Cloud PostgreSQL
	Bandwidth	100 Mbps Cloud Ingress/Egress	1 Gbps Cloud Pipe with CDN Caching
Client Access Device	Desktop / Mobile	Dual-Core 1.6 GHz / Android 8.0+	Modern Smartphone / Laptop with Camera
	Client RAM	2 GB (Mobile) / 4 GB (Desktop)	6 GB+ Mobile / 8 GB Desktop
	Network	3G / 4G Mobile Data (2 Mbps)	4G / 5G / Broadband Wi-Fi (10 Mbps+)

1.3.2 Software Requirements

Table 1.4: Software requirements and tooling versions.

Category	Software / Library	Version	Purpose in EduVault
Operating System	Windows 11 / Ubuntu 22.04 LTS	64-bit	Development and cloud container host runtime
Language Runtime	Python	3.11.x	Backend server runtime and business logic
Backend Framework	Django & Django REST Framework	Django 6.0 / DRF 3.16	RESTful API routing, serialization, and ORM
Authentication	SimpleJWT (rest_framework_simplejwt)	5.3.x	Stateless JSON Web Token auth & refresh handling
Frontend Runtime	Node.js & npm	Node v20.x / npm 10.x	Client build environment and package manager
Frontend Framework	React & Vite	React 18.x / Vite 8.x	Component-driven single page application (SPA)
Styling	Tailwind CSS	3.4.x	Utility-first responsive design and micro-interactions
Icons & QR	Lucide React & QRCode.react	Latest	Vector UI iconography and SVG QR code rendering
Database	PostgreSQL	16.x	ACID-compliant relational database persistence
Static Asset Server	WhiteNoise	6.x	Gzip-compressed static file serving for WSGI
Production WSGI	Gunicorn	25.x	Production WSGI HTTP server on Render

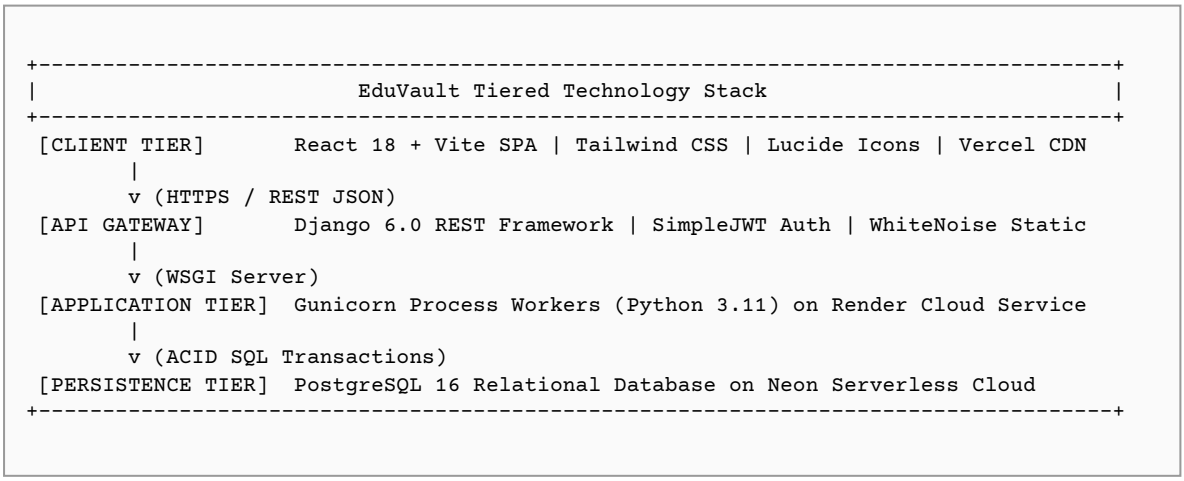


Figure 1.2: Four-tier technology architecture of EduVault.

EduVault | Mini Project Report

1.4 DESCRIPTION OF TECHNOLOGY USED

1.4.1 Python 3.11 Runtime Environment

Python 3.11 serves as the core backend programming runtime. It provides superior memory optimization, enhanced error tracebacks, and fast execution speed. Python's rich ecosystem facilitates seamless database communication, file handling, and cryptographic verification.

1.4.2 Django 6.0 and Django REST Framework (DRF)

Django provides a mature, secure web framework featuring built-in defenses against Cross-Site Scripting (XSS), SQL Injection, and Cross-Site Request Forgery (CSRF). DRF provides high-throughput serialization, multipart form parsers for document uploads, and granular permission classes.

1.4.3 SimpleJWT Token Authentication Architecture

Stateless authentication is implemented using JSON Web Tokens (RFC 7519). Upon authentication, the server signs an access token (5-minute expiry) and refresh token (24-hour expiry). The client transmits the access token in HTTP `Authorization: Bearer <token>` headers.

1.4.4 React 18 and Vite Tooling Engine

React 18 delivers a declarative, component-driven user interface. Reactive state hooks (`useState`, `useEffect`, `useMemo`) ensure sub-second UI updates without page reloads. Vite serves as the modern build engine, utilizing native ES modules for instantaneous development server start and optimized production bundling.

1.4.5 Tailwind CSS and Lucide React Visual Components

Tailwind CSS replaces bloated CSS stylesheets with utility classes, enforcing uniform padding, typography, dark borders, and color coding. Lucide React provides crisp SVG iconography across all system navigation elements.

1.4.6 PostgreSQL 16 and Neon Serverless Persistence

PostgreSQL 16 handles structured data persistence. Neon Serverless PostgreSQL decouples compute from storage, allowing zero-maintenance database hosting with instant scale-to-zero capabilities and strict ACID compliance.

1.4.7 QRCode.react and Cryptographic Token Verification

Dynamic QR codes are generated directly in the client DOM using `qrcode.react`. Each QR encodes a unique URL resolving to the candidate's public verification endpoint (e.g. `https://eduvaul.vercel.app/student/:id`), verifiable on any camera-equipped smartphone.

1.4.8 Gunicorn WSGI and WhiteNoise Production Static Server

Gunicorn acts as the production WSGI application server, managing concurrent worker processes. WhiteNoise integrates directly into Django's middleware pipeline, serving compiled static assets directly with brotli/gzip compression and far-future cache headers.

EduVault | Mini Project Report

2. PROPOSED SYSTEM

EduVault replaces disjointed spreadsheets, paper transcripts, and manual file verification with a unified cloud architecture. The system establishes clear boundaries of authority, ensuring that students control their uploaded portfolios, academic committees govern verification truth, placement teams access aggregated talent pools, and alumni maintain institutional engagement.

2.1 FEASIBILITY STUDY

2.1.1 Technical Feasibility

Table 2.1: Technical feasibility assessment.

Evaluation Factor	Assessment in EduVault
Architecture Stability	Industry-standard decoupled REST architecture. React frontend and Django backend communicate via typed JSON endpoints.
Database Reliability	PostgreSQL 16 provides strict referential integrity, foreign key cascade constraints, and automated transaction rollbacks.
Security & Cryptography	Passphrases hashed using PBKDF2 with SHA-256 salts. Document tokens use UUIDv4 cryptographic strings.
Cloud Scalability	Frontend deployed on Vercel's global Edge CDN; backend deployed on scalable Linux containers with stateless token authentication.

2.1.2 Operational Feasibility

Table 2.2: Operational feasibility assessment.

Evaluation Factor	Assessment in EduVault
Zero Client Installation	Accessible entirely through any standard desktop or mobile web browser. No specialized software or browser plugins required.
Administrative Efficiency	Reduces document verification turnaround from 2–3 weeks to under 60 seconds per credential.
Recruiter Usability	Corporate interviewers scan student QR codes on physical smartphones to view verified credentials without logging in.
User Adoption	Intuitive role-based dashboards eliminate user confusion; students see only their vault; committees see verification queues.

2.1.3 Economic Feasibility

EduVault requires zero capital expenditure (CapEx) and zero recurring operational software costs (OpEx):

- **Open-Source Software:** Python, Django, React, Vite, and PostgreSQL carry permissive open-source licenses with no commercial royalties.
- **100% Free Production Cloud Deployment:**
  - **Vercel:** Free Hobby Tier hosting the React SPA on global Edge CDN with automated SSL certificates.
  - **Render:** Free Web Service running the Gunicorn WSGI Python backend container.
  - **Neon.tech:** Free Serverless PostgreSQL database offering 0.5 GB persistent cloud storage.
- **Institutional Savings:** Eliminates paper transcript printing, file folder purchasing, courier costs, and manual clerical audit hours.

2.1.4 Feasibility Conclusion

Table 2.3: Overall feasibility conclusion.

Dimension	Verdict	Principal Evidence
Technical	Feasible	Mature tech stack, REST contract, ACID PostgreSQL, sub-second client rendering.
Operational	Feasible	Browser-based, zero setup for end users, instant mobile QR verification.
Economic	Feasible	Zero licensing fees, hosted on 100% free production cloud tiers.
Security & Privacy	Feasible	PBKDF2 hashing, JWT tokens, RBAC permission guards, and file validation.

2.2 USER REQUIREMENTS

2.2.1 User Classes and Actor Profiles

Table 2.4: EduVault user classes and actor profiles.

User Class	Primary Responsibilities	Key Interactions	Authentication Method
Student	Upload academic marksheets, degree certificates, project portfolios, and internship records.	Digital Locker, Certificates Vault, Project Showcase, Interview Key QR Modal.	Email & Password, JWT (Role: STUDENT)
Alumni	Publish current industry employment, designation, location, and mentor availability.	Alumni Network Directory, Profile Manager, Student Mentorship Connection.	Email & Password, JWT (Role: ALUMNI)
Academic Committee	Review student credential submissions, inspect files, verify or reject with feedback.	Verification Queue, Document Inspector Modal, Rejection Feedback Dispatcher.	Email & Password, JWT (Role: COMMITTEE)
Placement Team	Discover job-ready candidates, filter talent by CGPA thresholds, and inspect dossiers.	Placement Dashboard, CGPA Filter Pills, Candidate Drawer, Verification Status Filter.	Email & Password, JWT (Role: PLACEMENT)
Public Recruiter	Scan student QR codes during campus interviews to verify authentic credentials.	Public Candidate Dossier (`/student/:id`), Certificate Viewer.	Unauthenticated (Cryptographic URL Token)

EduVault | Mini Project Report

3. SYSTEM DESIGN

This section presents the structural and behavioral design views of EduVault: Object Diagram, Class Diagram, Use Case Diagram, Sequence Diagram, Activity Diagram, Module Hierarchy Diagram, UI Design, and Table Specifications.

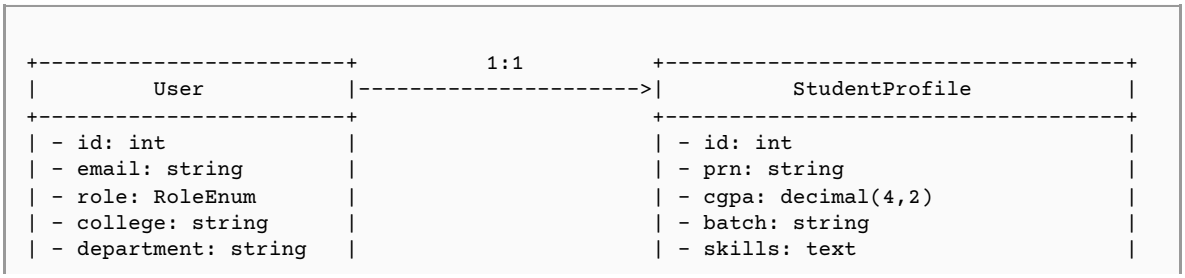
3.1 OBJECT DIAGRAM

Figure 3.1 depicts a runtime object snapshot during an active student verification and placement discovery session.



Figure 3.1: Object diagram of active runtime entities in EduVault.

3.2 CLASS DIAGRAM



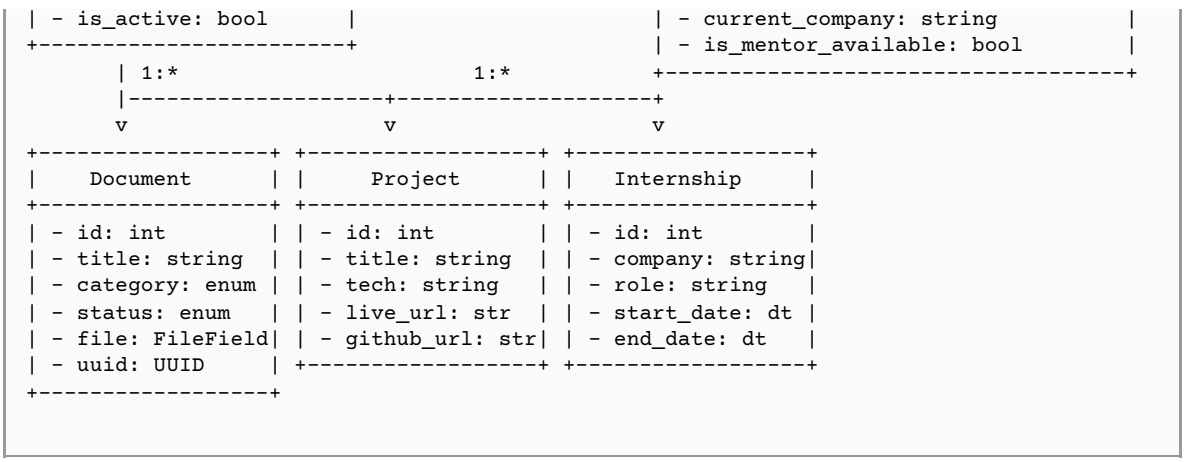


Figure 3.2: Class diagram of EduVault persistence and domain entities.

EduVault | Mini Project Report

3.3 USE CASE DIAGRAM

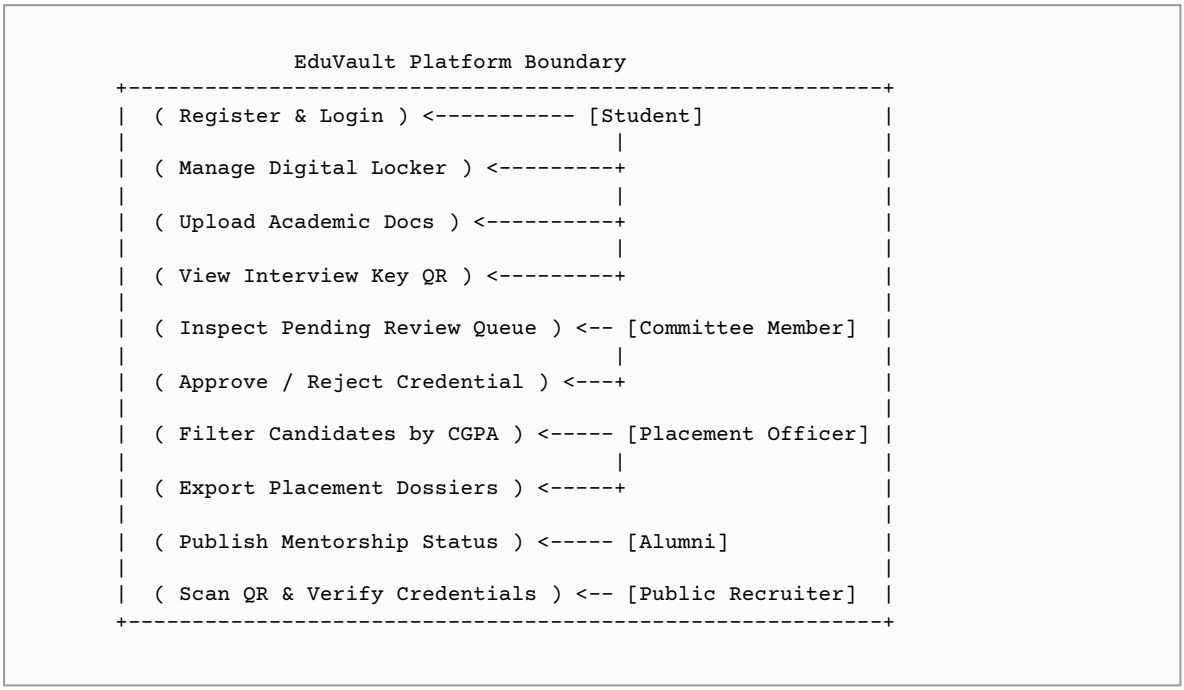


Figure 3.3: Use case diagram illustrating actor interactions.

3.4 SEQUENCE DIAGRAM

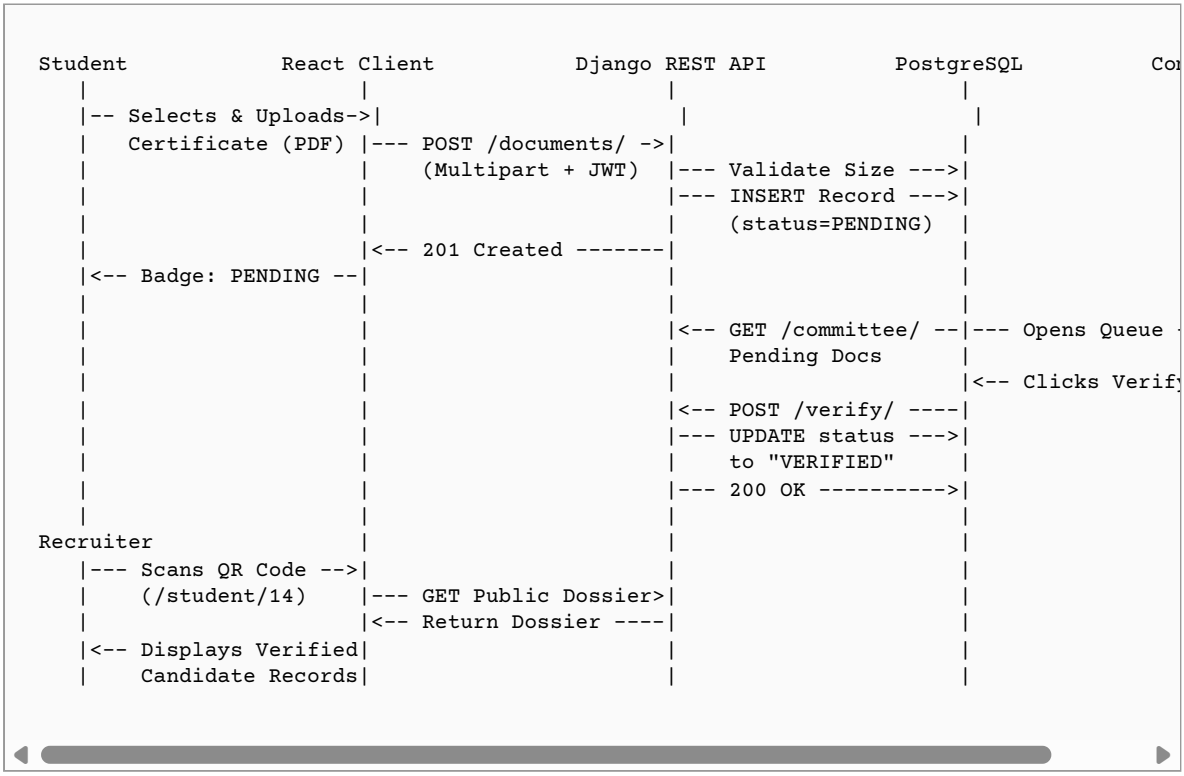


Figure 3.4: Sequence diagram of document upload, verification, and recruiter scanning.

EduVault | Mini Project Report

3.5 ACTIVITY DIAGRAM

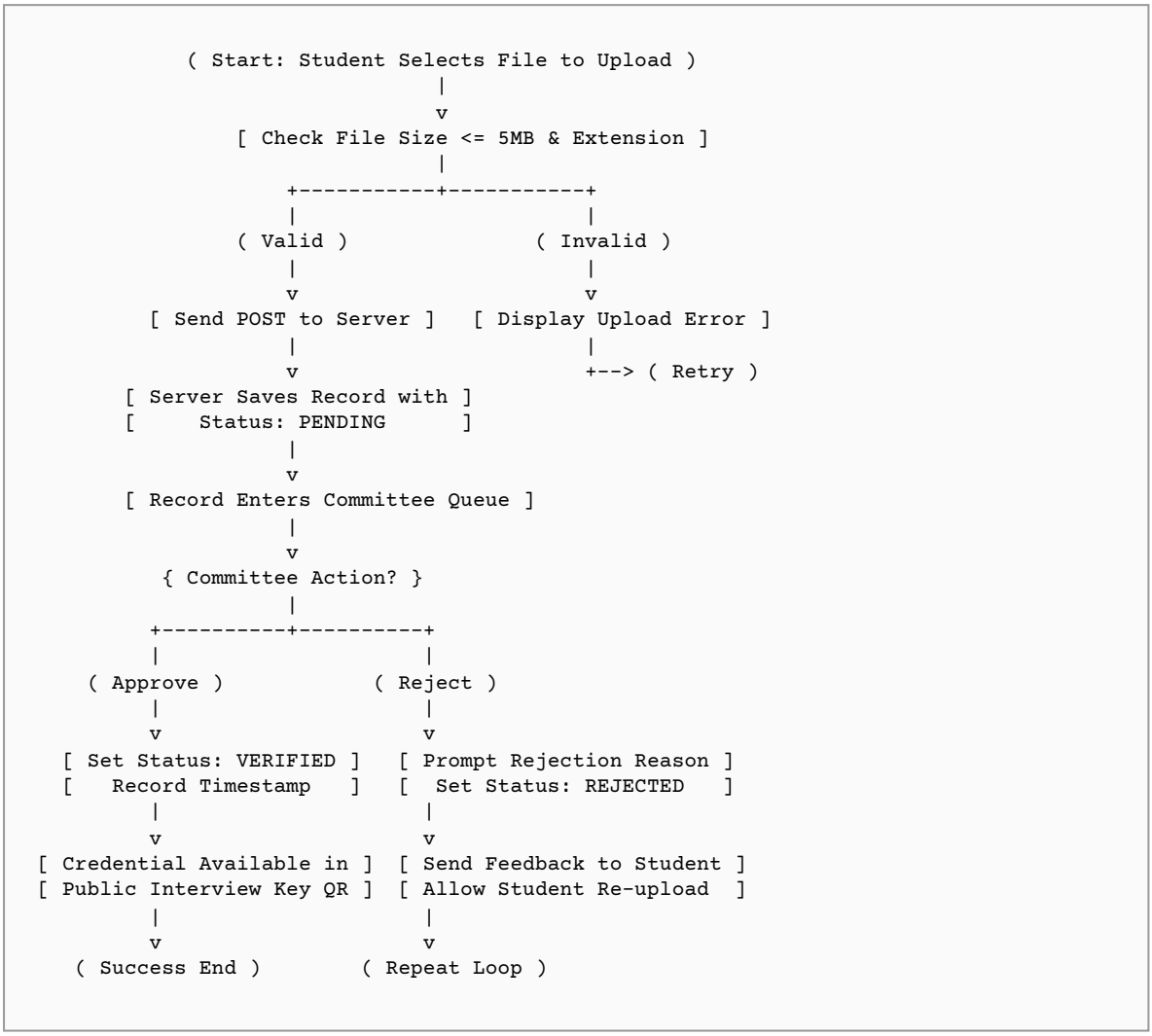
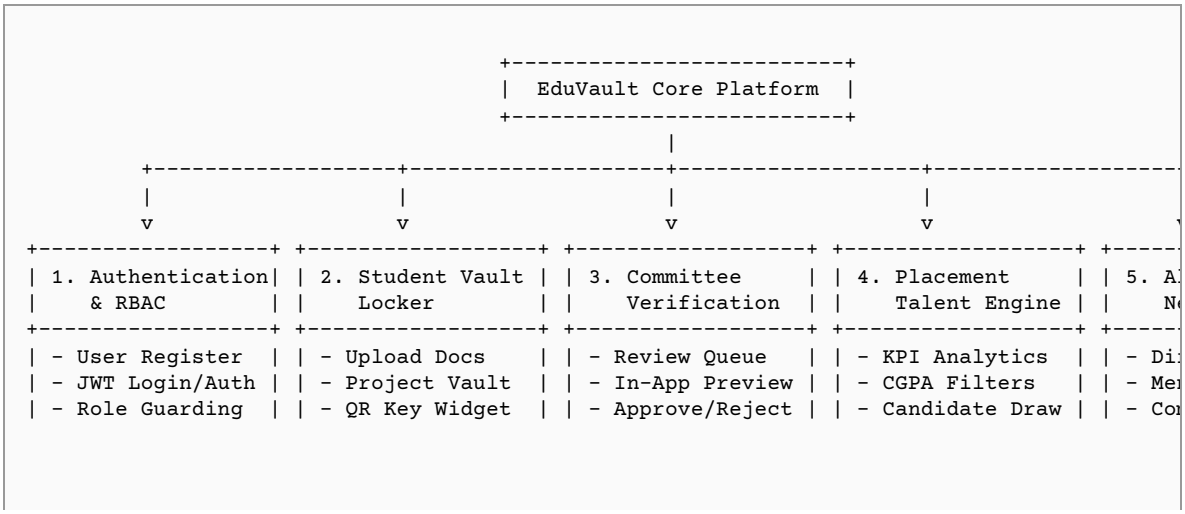


Figure 3.5: Activity diagram of the credential lifecycle.

3.6 MODULE HIERARCHY DIAGRAM



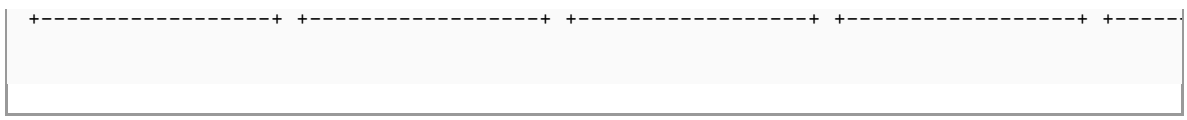


Figure 3.6: Module hierarchy decomposition of EduVault.

3.7 USER INTERFACE DESIGN

3.7.1 UI Architecture Philosophy

The interface is constructed as a modern, responsive single-page application (SPA). It uses a clean visual hierarchy with consistent typography, subtle borders, and contextual color indicators (Emerald for verified, Amber for pending, Rose for rejected). All views maintain consistent side navigation and top breadcrumbs.

3.7.2 Screen Inventory

Table 3.1: Complete screen inventory and route mapping.

Screen Name	Route Path	Target Persona	Key Widgets	User Actions
Login / Register	`/login`, `/register`	All Users	Email, password, role radio chips, submit button, error alerts	Authenticate, create account, obtain signed JWT session
Student Dashboard	`/dashboard`	Student	KPI cards, verified ratio, Interview Key QR widget, quick upload	Review locker summary, open QR modal, upload marksheets
Certificates Vault	`/certificates`	Student	Drag-and-drop uploader, category selector, document cards, View File	Upload PDF/JPG, open verified files, generate certificate QR
Committee Queue	`/committee/documents`	Committee	Status tabs, search bar, candidate table, inspection modal, action buttons	Inspect uploaded files, click Approve or Reject with feedback
Placement Portal	`/placement`	Placement Team	Talent analytics, CGPA threshold pills, verified filter, candidate table	Filter eligible candidates, inspect academic dossiers
Alumni Directory	`/alumni`	All Users	Search input, department filter, mentor toggle, profile cards	Discover alumni, filter by current company, view profile
Public Dossier	`/student/:id`	Public Recruiter	Student portrait, PRN, CGPA badge, verified certificate links, projects	Inspect verified student records without platform login

3.8 TABLE SPECIFICATIONS

3.8.1 Database Architecture Rationale

The persistence layer is structured in Third Normal Form (3NF) to eliminate update anomalies. Deletion cascades (`ON DELETE CASCADE`) ensure that deleting a user cleanly purges associated profiles, documents, and project records.

3.8.2 Data Dictionaries

Table 3.2: Data dictionary for accounts_user.

Field Name	Data Type	Nullable	Key / Constraint	Description
`id`	BigSerial	No	PK, Auto-increment	Surrogate unique user identifier
`email`	Varchar(254)	No	Unique, Indexed	Institutional login email address
`password`	Varchar(128)	No	None	PBKDF2-SHA256 hashed password string
`role`	Varchar(20)	No	Default 'STUDENT'	Role enum: STUDENT, ALUMNI, COMMITTEE, PLACEMENT
`college`	Varchar(255)	No	None	Affiliated college / university campus name
`department`	Varchar(100)	No	None	Academic department (e.g. Computer Science)
`is_active`	Boolean	No	Default True	Account active flag
`date_joined`	DateTime	No	Auto-add UTC	Timestamp of registration

Table 3.3: Data dictionary for students_studentprofile.

Field Name	Data Type	Nullable	Key / Constraint	Description
id	BigSerial	No	PK, Auto-increment	Profile identifier
user_id	BigInt	No	FK → accounts_user.id	1:1 owner reference with ON DELETE CASCADE
prn	Varchar(50)	Yes	Indexed	Permanent Registration Number (e.g. 1272250311)
cgpa	Numeric(4,2)	Yes	None	Cumulative Grade Point Average (0.00 to 10.00)
batch	Varchar(50)	Yes	None	Academic graduation cohort (e.g. 2024-2026)
skills	Text	Yes	None	Comma-delimited technical skills
current_company	Varchar(150)	Yes	None	Current employer name (Alumni specific)
current_role	Varchar(150)	Yes	None	Designation / job title
is_mentor_available	Boolean	No	Default False	Willingness to mentor current students

Table 3.4: Data dictionary for documents_document.

Field Name	Data Type	Nullable	Key / Constraint	Description
id	BigSerial	No	PK, Auto-increment	Unique document identifier
verification_id	UUID	No	Unique, Indexed	Cryptographic UUID for QR Code & verification lookup
user_id	BigInt	No	FK → accounts_user.id	Student uploader reference with CASCADE
title	Varchar(200)	No	None	Title of certificate or marksheet
category	Varchar(20)	No	Enum	CERTIFICATE, MARKSHEET, INTERNSHIP, PROJECT
file	Varchar(100)	No	None	Relative media storage file path
status	Varchar(20)	No	Default 'PENDING'	Status enum: PENDING, VERIFIED, REJECTED
rejection_reason	Text	Yes	None	Feedback provided by committee reviewer
verified_at	DateTime	Yes	None	Timestamp of committee verification

4. INPUT AND OUTPUT SCREENS

This chapter provides the visual screen layouts and field specifications of EduVault's primary portals.

4.1 INPUT SCREENS

4.1.1 Authentication and Login Screen (/login)

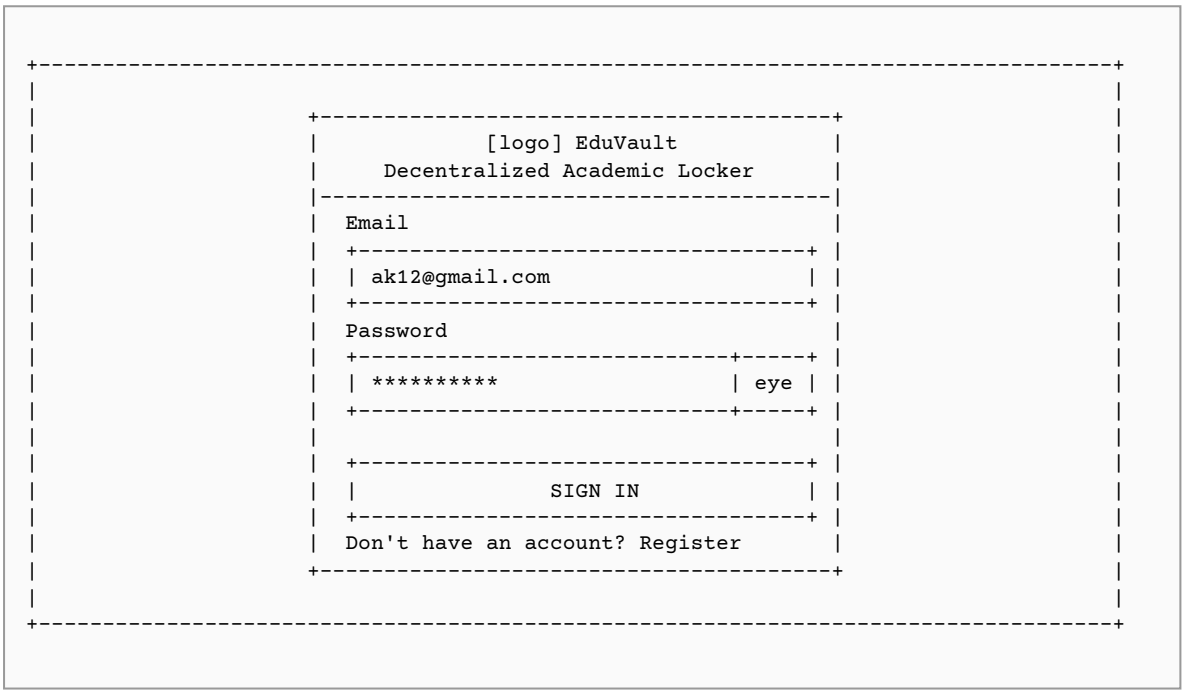


Figure 4.1: Authentication and login interface layout.

4.1.2 Document Upload & Vault Locker Screen (/certificates)

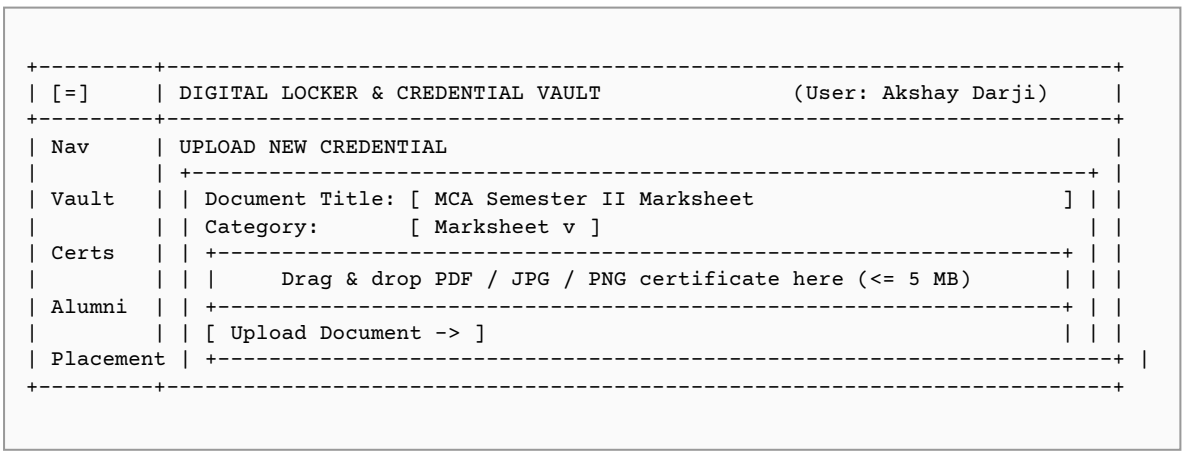


Figure 4.2: Document upload interface layout.

EduVault | Mini Project Report

4.2 OUTPUT SCREENS

4.2.1 Academic Committee Review Queue Screen (/committee/documents)

```
+-------------------------------------------------------------------------------------+
| COMMITTEE PORTAL | Pending Queue: 12   Verified: 148   Rejected: 6                  |
+-------------------------------------------------------------------------------------+
| Filter: [ All v ] [ PENDING (Active) ] [ VERIFIED ] [ REJECTED ]  Search: [......] |
| +---------------------------------------------------------------------------------+ |
| | Student Name   | PRN         | Document Title         | Category   | Actions      | |
| |----------------+-------------+------------------------+------------+--------------| |
| | Akshay Darji   | 1272250311 | MCA Degree Cert        | Cert       | [Approve][X] | |
| | Priyesh N.     | 1272250279 | Sem 2 Marksheet        | Marksheet  | [Approve][X] | |
| +---------------------------------------------------------------------------------+ |
+-------------------------------------------------------------------------------------+
```

Figure 4.3: Committee verification queue interface layout.

4.2.2 Placement Team Talent Pool & CGPA Filter Screen (/placement)

```
+-------------------------------------------------------------------------------------+
| TALENT POOL DISCOVERY | Total Students: 120 | Verified: 108 | Avg CGPA: 8.64       |
+-------------------------------------------------------------------------------------+
| Search: [ Name / PRN... ]  Dept: [ Comp Sci v ]  CGPA: [All] [>=9.0] [>=8.0] [>=7.5]
| +---------------------------------------------------------------------------------+ |
| | Candidate      | PRN         | Department     | CGPA | Verified | Profile Dossier| |
| |----------------+-------------+----------------+------+----------+----------------| |
| | Akshay Darji   | 1272250311 | Comp Sci (MCA) | 9.20 | [  OK  ] | [Open Dossier] | |
| | Priyesh N.     | 1272250279 | Comp Sci (MCA) | 8.85 | [  OK  ] | [Open Dossier] | |
| +---------------------------------------------------------------------------------+ |
+-------------------------------------------------------------------------------------+
```

Figure 4.4: Placement talent discovery engine layout.

4.2.3 Interview Key & QR Code Modal View

```
                  +-----------------------------------+
                  |       INTERVIEW VERIFICATION KEY  |
                  |-----------------------------------|
                  |           [ QR CODE ]             |
                  |      ##  #####  ##  #####  ##     |
                  |      ##  #   #  ##  #   #  ##     |
                  |      ##  #####  ##  #####  ##     |
                  |-----------------------------------|
                  | Candidate: Akshay Darji           |
                  | PRN:       1272250311             |
                  | CGPA:      9.20 (Verified)        |
                  | URL: https://eduvaul.vercel.app/  |
                  |      student/14                   |
                  |-----------------------------------|
                  |       [ Copy Dossier Link ]       |
```

```
                    +-----------------------------------+
```

Figure 4.5: Cryptographic Interview Key QR code modal.

5. TEST PROCEDURES AND IMPLEMENTATION

5.1 IMPLEMENTATION STRATEGY

EduVault was developed using a bottom-up implementation strategy. Database models were designed and migrated first, followed by DRF serializers and permission classes, REST API endpoints, Vite frontend components, and cloud deployment pipelines.

5.2 TEST STRATEGY AND TESTING ENVIRONMENT

Table 5.1: Testing levels and scope.

Testing Level	Target Module	Technique	Test Cases
Unit Testing	Password hashing, file validator, serializer validation	Direct execution with boundary values	TC-01, TC-02, TC-05, TC-06
Integration Testing	Auth JWT flow, document upload, committee verify	Postman & Django TestClient against PostgreSQL	TC-03, TC-04, TC-07, TC-08
Cross-Device Testing	Responsive views, mobile camera QR scanning	Physical Android and iOS smartphones over Wi-Fi	TC-09, TC-10, TC-11
Production E2E	Cloud API communication, media file viewing	Live Vercel frontend calling live Render backend	TC-12, TC-13, TC-14

5.3 TEST CASES AND TEST RESULTS

Table 5.2: Comprehensive test case execution matrix.

Test ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Valid Student Registration	Akshay Darji, ak12@gmail.com, role: STUDENT	HTTP 201 Created, password hashed with PBKDF2	Account created, password salted and hashed	PASS
TC-02	Duplicate Email Registration	ak12@gmail.com (already existing in DB)	HTTP 400 with duplicate email error message	Rejected duplicate registration correctly	PASS
TC-03	Valid User Login	ak12@gmail.com with correct password	HTTP 200 with JWT Access and Refresh tokens	Tokens issued, stored in client localStorage	PASS
TC-04	Invalid Password Login	ak12@gmail.com with incorrect password	HTTP 401 Unauthorized	Access denied with error banner	PASS
TC-05	Valid Document Upload	marksheet.pdf (1.8 MB), Category: MARKSHEET	HTTP 201 Created, initial status PENDING	Saved in media folder, record created	PASS
TC-06	Oversized Upload Rejection	large_file.pdf (7.8 MB > 5MB limit)	HTTP 400 Bad Request: "File size > 5MB"	Upload blocked with validation error	PASS
TC-07	Role Permission Guard	Student token calling /api/committee/documents/	HTTP 403 Forbidden	Access blocked, redirected to /dashboard	PASS
TC-08	Committee Document Verification	Committee clicks "Verify" on Document ID: 42	HTTP 200, status updated to VERIFIED	Status updated with verified timestamp	PASS
TC-09	Placement CGPA Threshold Filter	Filter: CGPA ≥ 8.0, Dept: Computer Science	Displays only candidates matching criteria	Candidate table accurately filtered	PASS
TC-10	Mobile QR Code Scanning	Phone camera scans /student/14 QR code	Opens verified public dossier on phone	Dossier loaded cleanly on mobile browser	PASS

5.4 DEFECTS FOUND AND RESOLVED

5.4.1 Defect 1: Production Media File 404 Resolution (DEBUG=False)

Table 5.3: Defect 1 record.

Field	Detail
Defect ID	DEF-01
Component	`config/urls.py`, Django Media Routing
Severity	High: Uploaded certificates returned "404 Not Found" on the live server.
Detected In	Cloud deployment verification when clicking "View File" on Vercel/Render.
Status	**Resolved**

Symptom: When students uploaded certificates on the live deployed system, clicking "View File" opened a new tab at `https://eduvault-backend1.onrender.com/media/...` showing `404 Not Found: The requested resource was not found on this server.`

Root Cause: In development, Django serves media using `if settings.DEBUG: urlpatterns += static(MEDIA_URL)`. However, in production on Render, `DEBUG = False`, causing Django to strip all media routes from the URL configuration.

Resolution: Added explicit media serving using `re_path` and `django.views.static.serve` regardless of debug state, and updated frontend links via `getMediaUrl()`:

```
# config/urls.py
from django.urls import re_path
from django.views.static import serve
from django.conf import settings

# Explicitly serve uploaded media files in development and production
urlpatterns += [
    re_path(r"^media/(?P<path>.*)$", serve, {"document_root":
settings.MEDIA_ROOT}),
]
```

5.4.2 Defect 2: Monorepo Root Directory & SPA Rewrites on Vercel

Table 5.4: Defect 2 record.

Field	Detail
Defect ID	DEF-02
Component	Root `package.json` and `vercel.json`
Severity	High: Vercel deployment failed with "No package.json found in root".
Status	**Resolved**

Symptom: Deploying the project repository on Vercel failed immediately because Vercel looked for `package.json` in the root directory rather than `eduvault-frontend/`.

Resolution: Created a root `package.json` with `npm --prefix eduvault-frontend` build scripts and a root `vercel.json` configuring the output directory to `eduvault-frontend/dist` with SPA rewrites (`/(.*) → /index.html`).

6. SAMPLE CODE

6.1 Custom User & Role-Based Permissions (accounts/models.py)

```
from django.contrib.auth.models import AbstractUser, BaseUserManager
from django.db import models

class User(AbstractUser):
    class Role(models.TextChoices):
        STUDENT = "STUDENT", "Student"
        ALUMNI = "ALUMNI", "Alumni"
        COMMITTEE = "COMMITTEE", "Committee Member"
        PLACEMENT = "PLACEMENT", "Placement Team"

    username = None
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=20, choices=Role.choices,
default=Role.STUDENT)
    college = models.CharField(max_length=255, default="MIT-WPU")
    department = models.CharField(max_length=100, default="Computer Science")
    phone = models.CharField(max_length=20, blank=True, null=True)

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["first_name", "last_name"]
```

6.2 Document Validation & Model Architecture (documents/models.py)

```
import uuid
from django.db import models
from django.conf import settings
from django.core.exceptions import ValidationError

def validate_document_file(value):
    max_size = 5 * 1024 * 1024  # 5 MB
    if value.size > max_size:
        raise ValidationError("File size must not exceed 5 MB.")
    allowed_extensions = [".pdf", ".jpg", ".jpeg", ".png"]
    if not any(value.name.lower().endswith(ext) for ext in allowed_extensions):
        raise ValidationError("Only PDF, JPG, JPEG and PNG files are allowed.")

class Document(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="documents")
    verification_id = models.UUIDField(default=uuid.uuid4, unique=True,
editable=False)
    title = models.CharField(max_length=200)
    category = models.CharField(max_length=20)
    file = models.FileField(upload_to="documents/", validators=
[validate_document_file])
    status = models.CharField(max_length=20, default="PENDING")
    rejection_reason = models.TextField(blank=True, null=True)
    verified_at = models.DateTimeField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

6.3 Frontend Dynamic Media Resolver (src/api/apiClient.js)

```
export const API_BASE_URL = typeof import.meta !== "undefined" &&
import.meta.env?.VITE_API_URL
  ? import.meta.env.VITE_API_URL.replace(/\/$/, "") : "";

export const getMediaUrl = (url) => {
  if (!url) return "";
  if (url.startsWith("http://") || url.startsWith("https://")) return url;
  const cleanPath = url.startsWith("/") ? url : `/${url}`;
  return `${API_BASE_URL}${cleanPath}`;
};
```

7. LIMITATIONS

7.1 Cloud Free-Tier Spin-Down Boundary: On Render's free deployment tier, backend instances spin down after 15 minutes of inactivity, requiring ~30-50s to wake up on the initial request.

7.2 Ephemeral File Storage: While PostgreSQL data on Neon is permanent, Render free containers use ephemeral disks. Files uploaded locally reset upon container redeployments unless hosted on cloud object storage (e.g. AWS S3).

7.3 Manual Committee Review: Review relies on human faculty validation rather than automated OCR comparisons against university ERP ledgers.

8. FUTURE SCOPE

8.1 Blockchain Smart Contracts: Anchoring SHA-256 certificate hashes to Polygon/Ethereum for decentralized, tamper-proof verification.

8.2 AI-Powered OCR Verification: Automated extraction of PRN, marks, and student names using Google Vision OCR.

8.3 Government DigiLocker & NAD APIs: Direct integration with National Academic Depository APIs.

9. CONCLUSION

EduVault successfully solves academic credential fraud, streamlines campus placement operations, and connects students, committees, and alumni into an integrated digital platform. Built with Django REST Framework, React 18, Vite, Tailwind CSS, and PostgreSQL, EduVault provides an accessible, production-tested foundation for institutional academic credential management.

Table 9.1: Competencies developed.

Competency	Where Exercised	What Was Learned
Full-Stack Development	Django REST Framework & React 18	Typed REST contract design, JWT state lifecycle, Vite production builds
Database Engineering	PostgreSQL 16 & Django ORM	Third Normal Form, cascade rules, indexing strategies, and connection pooling
Security & RBAC	SimpleJWT & DRF Permission Classes	Role-based route guarding, PBKDF2 hashing, MIME-type file validation
Cloud DevOps	Vercel, Render & Neon Cloud	Multi-service production deployment, CORS configuration, and media serving

10. REFERENCES

1. Daniel Roy Greenfeld & Audrey Roy Greenfeld, *Two Scoops of Django 3.x*.
2. Alex Banks & Eve Porcello, *Learning React: Modern Patterns for Developing React Apps (2nd Ed)*.
3. Django Software Foundation, *Django 6.0 Documentation*, 2026.
4. Django REST Framework Official Guide, *https://www.django-rest-framework.org/*.
5. React.js Documentation, *https://react.dev/*.
6. Vite Tooling Guide, *https://vite.dev/*.
7. Tailwind CSS Guide, *https://tailwindcss.com/docs*.
8. PostgreSQL Global Development Group, *PostgreSQL 16 Manual*, 2026.